A Defect Detection and Closed-Loop Control System with MCGI-YOLO for Circular Knitting Machines

Kangzhao Tan, Jiarui Zhang, Junjie Yu, Heng Wu, Jing Lin*, Diyu Li*, and Chunhua He*

***Abstract*—Defect detection is critical for circular knitting production, where persistent needle or yarn abnormalities lead to repeated fabric defects, resulting in severe material waste and economic losses. Existing methods are rarely tailored to circular knitting machines and tend to be cost-prohibitive, limiting large-scale industrial deployment. To address this issue, we present a low-cost, dedicated defect detection and closed-loop control system for circular knitting machines, which triggers image acquisition from machine rotation, halts the machine when configurable stopping criteria are met, and alerts the operator for corrective intervention, forming a complete production–detection–control loop. To further improve detection accuracy for diverse fabric defect morphologies and scales, we propose a lightweight detector termed MCGI-YOLO. To model the morphological continuity of elongated defects, morphology-aware context encoding (MCE) augments backbone features with spatially extended context. To accommodate variations in defect morphology and scale, gated interpolation upsampling (GIU) adapts the reconstruction of higher-level context before fusion with local features, using a compact content-adaptive gate. On a real-world dataset collected from a circular-knitting production line, MCGI-YOLO achieves 97.2% mAP50 and 65.5% mAP50–95, improving on the YOLOv8n baseline by 6.7 and 7.1 percentage points, respectively. After INT8 conversion, the model reaches an inference throughput of 22 FPS on the edge device. The proposed system provides a cost-effective solution for real-time defect detection in circular-knitting production.**

***Index Terms*—Circular knitting machine, edge computing, fabric defect detection, gated interpolation upsampling (GIU), morphology-aware context encoding (MCE), YOLO.**

I. Introduction

CIRCULAR knitting machines are key production equipment for high-volume weft-knit fabric, which is widely used in everyday clothing such as T-shirts and sportswear [1]–[3]. The International Textile Manufacturers Federation reported approximately 24,500 global shipments of circular knitting machines in 2025 [4]. Globally, approximately 92 million tonnes of textile waste are generated annually, highlighting the importance of reducing avoidable material losses [5]. Conventional quality checks are often performed after knitting is completed. A persistent needle or yarn abnormality can therefore generate repeated defects along the fabric before discovery, accumulating substantial fabric waste and economic losses [6]–[8]. To prevent such losses, real-time detection during knitting is needed to identify these defects early and enable timely intervention before further defective fabric is produced.

Edge computing offers a practical pathway toward such real-time detection, and has been increasingly adopted for industrial defect detection. In textile applications, Machado et al. deployed YOLO11n on a Jetson Orin Nano to detect solid-color cotton and linen fabrics on a fabric rolling line [9]. Yan et al. deployed an improved YOLOv5s model on a Huawei Ascend 310-based Atlas platform to detect warp-knitted lace on warp-knitting machines [10]. Beyond textile detection, Wang et al. and Zhang et al. deployed an improved YOLOv5s model for rail surface defect detection and an improved Faster R-CNN detector with convolutional feature reuse for UAV-based insulator defect detection, respectively, both on Jetson Xavier NX [11], [12]. Li et al. deployed LiteYOLO-ID on a Jetson TX2 NX for insulator defect detection [13], and Zhao et al. implemented KAM-Net on a Xilinx ZCU102 FPGA for glass defect segmentation [14]. These studies collectively demonstrate the applicability of edge computing to industrial defect detection in their respective scenarios. However, such platforms are not tailored to the sensing, control, and installation constraints of circular knitting machines. The relatively high cost of general-purpose computing platforms, together with the additional integration effort required for image acquisition and machine control, hinders economically viable deployment across large numbers of machines.

In terms of detection algorithms, YOLO-based detectors have been widely explored for industrial defect detection because of their efficient inference and explicit defect localization [15]. FA-YOLO combines receptive-field enhancement, attention, and adaptive spatial feature fusion [16]. DGHR-YOLO integrates deformable convolution for morphology adaptation with a high-level screening feature pyramid and lightweight feature processing [17]. Zhao et al. proposed attention-based multiscale feature fusion (AMFF), combining intralayer feature enhancement and cross-layer attention to strengthen weak defect representations on fabric and aluminum

Manuscript received October 31, 2026; revised xxxx xx, xxxx; accepted xxxx xx, xxxx. This work was supported in part by the National Natural Science Foundation of China under Grant U22A2012. *(*Corresponding authors: Jing Lin, Diyu Li, and Chunhua He.)*

Kangzhao Tan, Jiarui Zhang, Junjie Yu, and Chunhua He are with the School of Computer Science, Guangdong University of Technology, Guangzhou 510006, China (e-mail: tankangzhao@mails.gdut.edu.cn; zhangjiarui1@mails.gdut.edu.cn; yujunjie2@mails.gdut.edu.cn; hechunhua@pku.edu.cn). Heng Wu is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China (e-mail: heng.wu@gdut.edu.cn). Jing Lin is with the Classroom Management Center, Guangdong University of Technology, Guangzhou 510006, China (e-mail: lj@gdut.edu.cn). Diyu Li is with Guangzhou Vinsome Automation Technology Co., Ltd., Guangzhou, China (e-mail: lidiyu@vinsome.cn).

surfaces [18]. An and Wan introduced JDCBL, combining contextual Transformer features with balanced contrastive learning to address scale variation and long-tailed defect categories in fabric detection [19]. Wang et al. developed RRNet, a lightweight rotated fabric defect detector that combines position-guided fusion and multiscale rotation-guided strip coupling to model elongated and tilted defect structures [20]. Together, these methods advance the representation of subtle defects, irregular shapes, and scale variation. Related work on photovoltaic-cell and pipeline-weld detection also explores task-specific feature pyramids, contextual relations, and lightweight rotated representations [21]–[23]. However, these methods are tailored to their respective detection targets and may not be directly applicable to defect detection on circular knitting machines. Defects on circular knitting machines exhibit diverse morphologies and scales, ranging from horizontal and vertical elongated patterns to localized holes. In addition, modules such as deformable convolution and attention often introduce substantial computational overhead, making them less suitable for deployment on resource-constrained edge devices.

Motivated by these limitations, we design and implement a low-cost, dedicated defect detection and closed-loop control system for circular knitting machines. The system integrates a power adapter, a global-shutter camera with LED illumination, a coaxial serializer/deserializer (SerDes) image-transmission link, a Hall sensor, and an edge device. For defect detection within this system, we develop MCGI-YOLO, a lightweight detector that incorporates morphology-aware and scale-aware design for fabric defects. MCE supplies spatially extended morphology-aware context for horizontal and vertical elongated defects. GIU addresses the differing reconstruction needs of elongated defects and holes of varying sizes by adaptively adjusting the interpolation of higher-level context. Together, the two components connect morphology-aware context encoding with spatial reconstruction within a compact parameter budget.

The main contributions of our work are as follows.

1) We design and implement a dedicated defect detection and closed-loop system for circular knitting machines, integrating image acquisition, camera transmission, edge inference, and local machine-stop control, and demonstrate its deployment on a real production line.
2) We develop MCE to explicitly incorporate the morphology priors of horizontal and vertical elongated defects. It embeds spatially extended, morphology-aware context in a partial C2f branch with small additional overhead.
3) We design GIU to accommodate the diverse morphologies and scales of circular-knitting defects, including elongated defects and holes of varying sizes. A compact spatial gate adapts a convex combination of two fixed interpolation paths, balancing response replication and neighborhood smoothing in the top-down path.

The remainder of this paper is organized as follows. Section II presents the defect detection and closed-loop control system. Section III describes MCGI-YOLO and its two components. Section IV defines the dataset and evaluation protocol. Section V reports detector comparisons and component ablation. Section VI presents model deployment, client and cloud operation, configurable stopping rules, and the analysis of delayed intervention. Section VII concludes.

II. Defect Detection and Closed-Loop Control System

A. System Overview

We design and implement a dedicated defect detection and closed-loop control system for circular knitting machines, as shown in Fig. 1. The closed-loop workflow begins when the operator starts the machine. The Hall sensor detects machine rotation and triggers the edge device to activate the camera and LED illumination for image acquisition and defect detection. When the detections in a frame satisfy the configured stopping criteria, the edge device immediately sends a stop command to the machine controller to prevent further defect propagation, uploads the defect image and record to the cloud, and pushes an alert notification to the operator. After the operator inspects the reported defect, resolves the production fault, and restarts the machine, the rotation signal reactivates detection. This sequence forms a complete production–detection–control loop. The system comprises power supply, image acquisition and transmission, processing, and network communication functions. The following subsections describe the hardware subsystems and their implementation. Model deployment, client and cloud interfaces, and stopping rules are presented in Section VI.

B. Power Supply Subsystem

To accommodate different industrial supply voltages across countries and regions, we design a power adapter with a 100–500 V AC input range and a 24 V DC output rated at 3 A. The adapter first supplies this output to the device. The device then supplies the LED illumination. By adjusting the current delivered to the LEDs, the device controls the illumination brightness to suit the image-acquisition conditions. This arrangement integrates LED power delivery and brightness control into the device.

C. Image Acquisition and Transmission Subsystem

The rapid rotation of circular knitting machines requires imaging that preserves defect geometry and fine spatial detail. We therefore select a SmartSens SC130GS global-shutter sensor (SmartSens Technology Co., Ltd., Shanghai, China) at 1280×1024 resolution, which exposes all pixels simultaneously to avoid motion-induced distortion from sequential row exposure. At a camera-to-fabric distance of approximately 10 cm, the field of view is approximately 16×12 cm, with the 16 cm dimension aligned with the horizontal direction of fabric motion.

Factory machinery creates an electromagnetically noisy environment, and installation requires the camera and edge device to be separated by a cable run. We therefore use serializer/deserializer (SerDes) transmission over shielded coaxial cable to accommodate remote camera placement and the interference environment. Specifically, the THCV241A serializer and THCV242A deserializer (THine Electronics, Inc., Tokyo, Japan) bridge the camera-side and processor-side MIPI

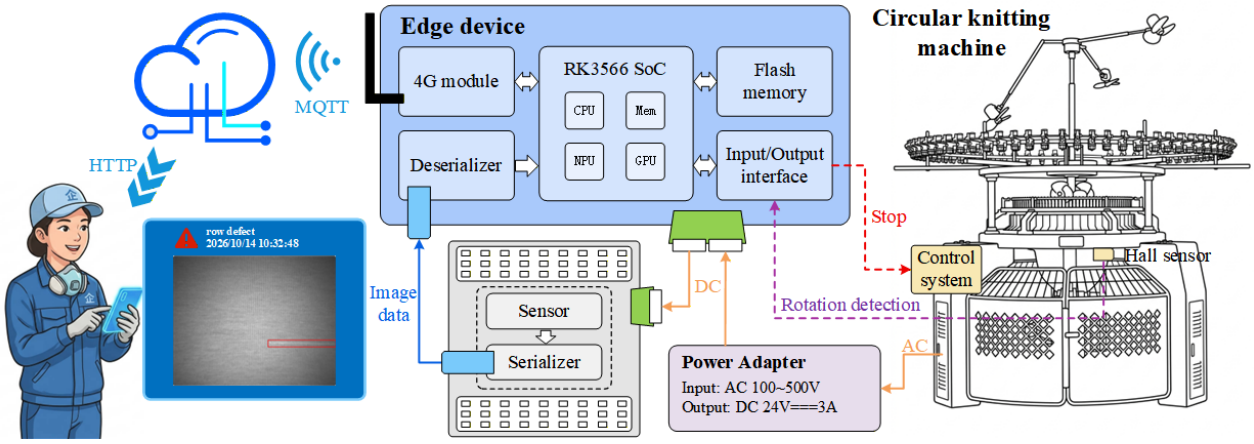

Fig. 1. The device comprises LED illumination control, a coaxial image link to the RK3566, 4G cloud connectivity, and I/O for rotation sensing and machine control.

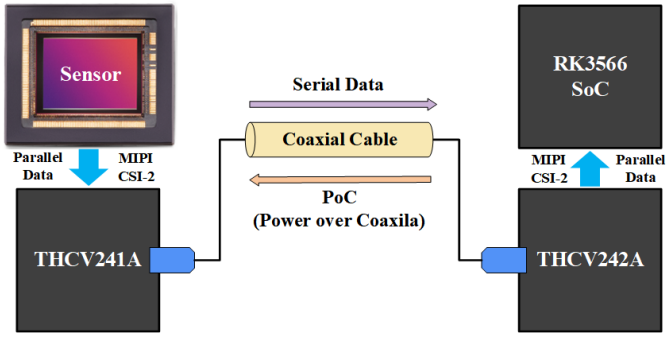

Fig. 2. SerDes camera link using the THCV241A serializer and THCV242A deserializer. Image data travel from the sensor to the RK3566 over coaxial cable, while PoC supplies power to the camera side.

CSI-2 interfaces. As illustrated in Fig. 2, the serializer converts the camera output into a serial stream, which is transmitted over the coaxial cable and recovered by the deserializer for the RK3566. Power over Coax (PoC) supplies the camera side through the same cable, combining image transmission and power delivery in a single connection. The link supports a maximum serial line rate of 4.0 Gb/s per lane, corresponding to a rated effective rate of 3.2 Gb/s [24], [25].

The camera is configured to transmit 1280×1024 RAW10 images at 30 frames/s. With $b = 10$ bits per pixel, the active-image payload rate is

$$\begin{aligned} R_{\text{img}} &= WHbf_{\text{cam}} \\ &= 1280 \times 1024 \times 10 \times 30 \\ &= 393.216 \text{ Mb/s} \end{aligned} \tag{1}$$

where W and H denote the image width and height, and f_{cam} is the camera acquisition rate. This corresponds to only 12.3% of the rated single-lane effective capacity, providing a stable image input for the detection system.

D. Processor Subsystem

The processor is selected according to the detection rate required by fabric motion and the camera field of view. Taking a typical circular knitting machine as an example, with a machine diameter of 40 in ($D = 1.016$ m) and a rotation period of $T = 2$ s, the fabric travels approximately $\pi D = 3.192$ m per revolution. With an effective field-of-view width of $w = 0.16$ m along the direction of motion, the required frame rate is

$$f_{\min} = \frac{\pi D}{Tw} = \frac{\pi \times 1.016}{2 \times 0.16} \approx 9.97 \text{ frames/s} \tag{2}$$

We set the effective detection-throughput requirement to 10 frames/s. For this throughput target, we select the Rockchip RK3566 (Rockchip Electronics Co., Ltd., Fuzhou, China) to balance processing capability, camera connectivity, and hardware cost. It integrates a quad-core Cortex-A55 CPU and an NPU rated at 1 TOPS for INT8 operations [26].

TABLE I
HARDWARE PLATFORMS USED IN RELATED DEFECT DETECTION STUDIES.

Platform	Compute capability	Price (USD)
NVIDIA Jetson TX2 NX [13]	1.33 TFLOPS (FP16)	249
NVIDIA Jetson Xavier NX [11], [12]	21 TOPS (INT8)	399
NVIDIA Jetson Orin Nano [9]	40 TOPS (INT8)	499
Huawei Ascend 310 [10]	16 TOPS (INT8)	550
AMD Xilinx ZCU102 [14]	3.45 TOPS (INT8)†	3234
Our device	1 TOPS (INT8)	60

†AMD reference INT8 peak for three B4096 DPU cores.

Table I summarizes the hardware specifications of platforms used in related defect detection studies. While high-performance platforms like the NVIDIA Jetson Orin Nano and Xilinx ZCU102 offer substantial computational power, their costs range from hundreds to over 3,000 USD, which can be prohibitive for large-scale deployment. In contrast, our device provides a cost-effective hardware basis for the proposed system at a hardware cost of 60 USD, offering a significant cost advantage over comparable industrial platforms and making it highly suitable for large-scale deployment in textile factories.

E. Network Communication Subsystem

The edge device connects to the cloud through a 4G module with a SIM-card slot and an external antenna interface. This connection supports defect-image and event uploads, alert notifications, device-state reporting, and remote client

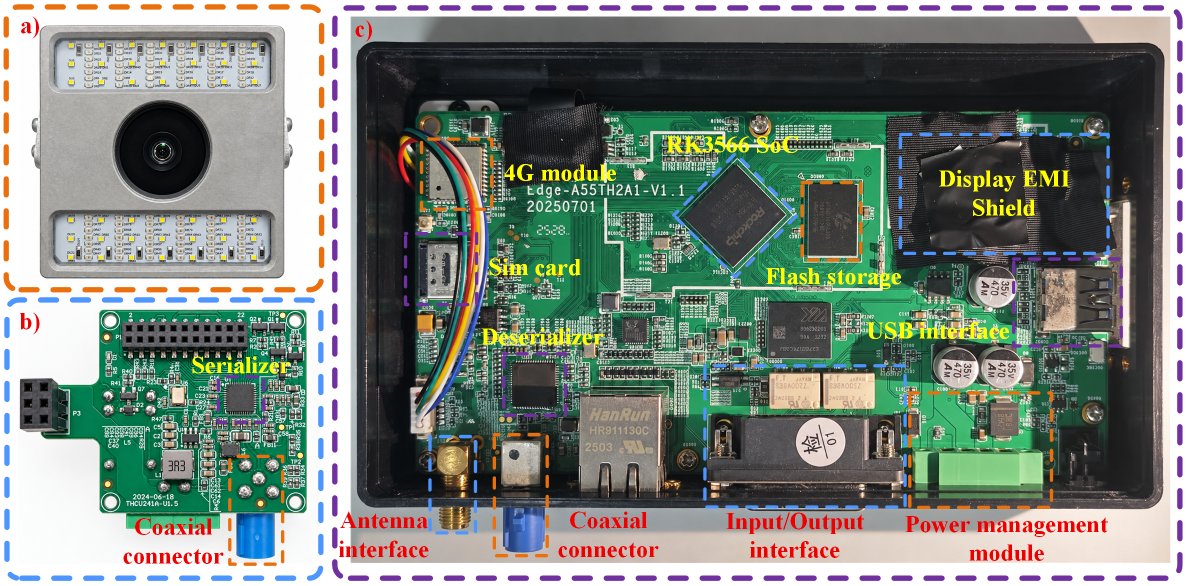

Fig. 3. Hardware of the defect detection system: (a) industrial camera with LED illumination; (b) camera-side interface board with a serializer and coaxial connector; and (c) RK3566 edge board with the 4G communication, flash storage, camera-interface, input/output interface, and power-management components.

configuration. The client and cloud functions are detailed in Section VI.

F. Hardware Implementation

Based on the design requirements of the subsystems described above, we develop the corresponding hardware shown in Fig. 3. The camera assembly (a) combines the image sensor and LED illumination. We design a camera-side interface board (b) that integrates the serializer and coaxial connector, and a mainboard (c) built around the RK3566. The mainboard integrates flash storage, the camera deserializer and coaxial connector, a 4G module with SIM-card and antenna interface, input/output interface, and power-management circuitry. This board-level integration brings image reception, local inference, network communication, machine control, and illumination control into a dedicated edge device. Fig. 4 shows the system installed on a circular knitting machine in a real production line.

III. Detection Algorithm

A. MCGI-YOLO Architecture

Fig. 5 shows MCGI-YOLO based on YOLOv8n [27]. Morphology-aware context encoding (MCE) is embedded in the P3 and P4 backbone C2f blocks to capture the continuity of horizontal and vertical elongated defects through spatially extended context. Gated interpolation upsampling (GIU) replaces the two top-down upsampling operations, adapting higher-level context reconstruction to different defect morphologies and scales before lateral feature fusion. The remaining backbone, bottom-up path, and detection head are retained [28], [29].

B. Morphology-Aware Context Encoding

MCE focuses on horizontal and vertical elongated defects produced by circular knitting machines. Elongated defects are characterized by a high aspect ratio. Their continuity spans across large spatial regions, making it difficult for local receptive fields to establish long-range dependencies. Local feature extraction alone fails to explicitly associate these spatially separated responses as parts of the same defect. Meanwhile, modeling long-range dependencies through dense channel-wise interactions would introduce substantial redundancy, imposing an impractical computational overhead on the RK3566. We therefore develop MCE to incorporate the morphology prior of elongated defects through spatially extended context within a compact computational budget. MCE is inserted after the last bottleneck in the selected C2f blocks, before branch concatenation and fusion [Fig. 6(a)]. The original split branches and intermediate bottleneck outputs remain separate inputs to concatenation; only the final bottleneck output is replaced by its MCE-refined counterpart.

1) Channel Allocation: As illustrated in Fig. 6(d), its input is $F \in \mathbb{R}^{B\times C\times H\times W}$. The channels are divided into two groups: one quarter of the channels, denoted by U, are used for morphology-aware context aggregation and residual modulation; the remaining three quarters, denoted by V, pass through unchanged. This design intentionally applies morphology-aware context aggregation to only a small channel subset, keeping the computational overhead compact. The two groups are concatenated after modulation, allowing the subsequent fusion to combine morphology-aware and unmodulated features with minimal performance loss. With $q = C/4$, the two groups have sizes $U \in \mathbb{R}^{B\times q\times H\times W}$ and $V \in \mathbb{R}^{B\times(C-q)\times H\times W}$. The split is performed along the channel dimension, preserving the batch and spatial dimensions.

2) Morphology-Aware Context Aggregation: The one-quarter-channel group U feeds the two pooling paths. Here, b, a, h, and w index the batch sample, channel, row, and column, respectively. Inspired by the pooling design of coordinate attention [30], we average independently for each sample and

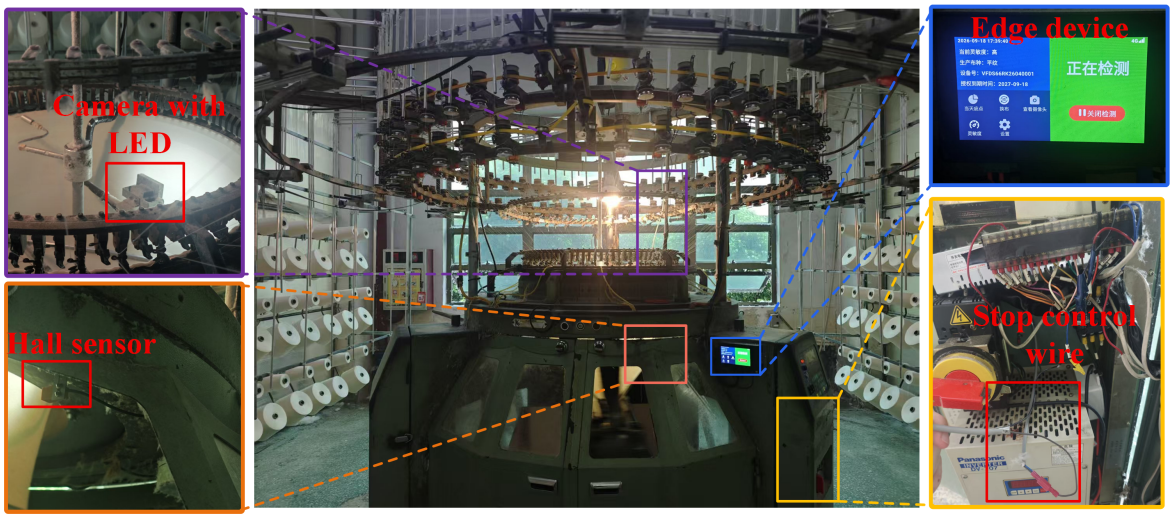

Fig. 4. Actual deployment on the circular knitting machine at Dongguan Minghai Textile Co., Ltd., Dongguan, Guangdong, China. The insets highlight the start button, Hall sensor, industrial camera, and RK3566 edge device, respectively.

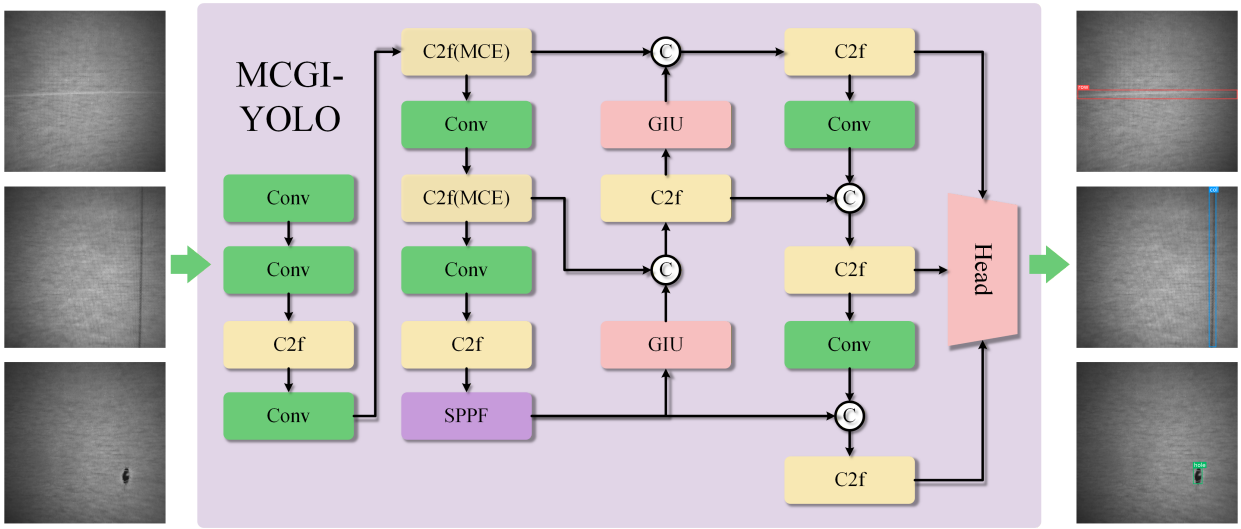

Fig. 5. Architecture of MCGI-YOLO. MCE operates within the stride-8 and stride-16 backbone C2f stages. GIU replaces the two top-down upsampling operations. The local bottlenecks, bottom-up aggregation path, and three-scale detection head are retained from YOLOv8n.

channel:

$$R_h(b,a,h,1) = \frac{1}{W}\sum_{w=1}^{W} U(b,a,h,w) \tag{3}$$

$$R_w(b,a,1,w) = \frac{1}{H}\sum_{h=1}^{H} U(b,a,h,w) \tag{4}$$

Averaging over W retains row positions, while averaging over H retains column positions. The descriptors are refined by two depthwise convolutions:

$$\begin{aligned} D_h &= \mathrm{DW}_{3\times1}(R_h) \\ D_w &= \mathrm{DW}_{1\times3}(R_w) \end{aligned} \tag{5}$$

The outputs D_h and D_w have sizes $B\times q\times H\times 1$ and $B\times q\times 1\times W$, respectively. Before addition, D_h is broadcast along W by repeating its column, while D_w is broadcast along H by repeating its row. The expanded features $\widetilde{D}_h$ and $\widetilde{D}_w$ both have size $B\times q\times H\times W$ and are added element-wise:

$$T = \widetilde{D}_h + \widetilde{D}_w. \tag{6}$$

Pooling provides long-range context, while the depthwise convolutions capture neighboring positions in the compressed descriptors.

3) Residual Context Modulation: The combined context T is mapped by Hardsigmoid and centered to construct a scaled residual correction:

$$G = \mathrm{HS}(T) \tag{7}$$

$$U' = U + \gamma \odot U \odot (2G-1) \tag{8}$$

where HS is Hardsigmoid, $\odot$ denotes element-wise multiplication with broadcasting, and $\gamma \in \mathbb{R}^{1\times q\times 1\times 1}$ is a learnable channel-wise parameter. The context response $G \in [0,1]^{B\times q\times H\times W}$ is centered by $2G-1$ to provide signed modulation, with γ controlling its channel-wise strength. The

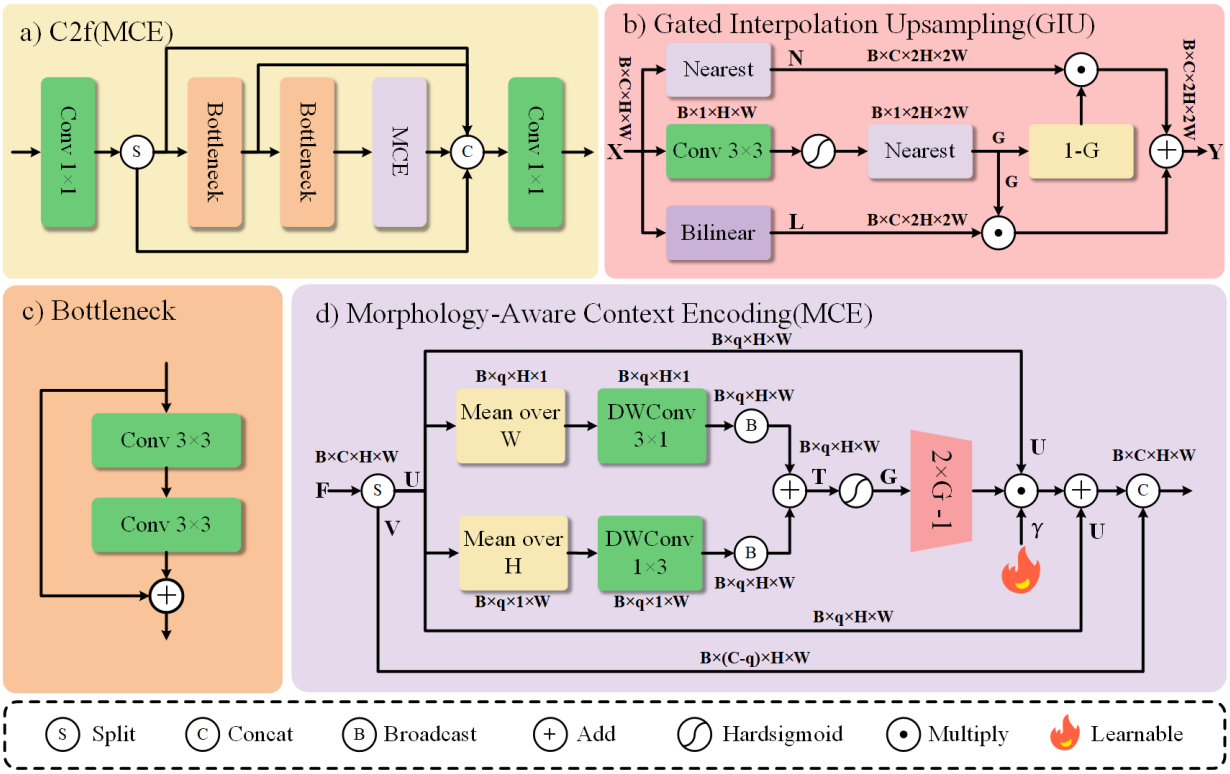

Fig. 6. Structures of the proposed modules: (a) C2f block with MCE; (b) GIU with a spatial weight map shared across channels; (c) residual bottleneck retained in C2f; and (d) MCE with context modulation on one quarter of its input channels and an unchanged bypass for the remaining three quarters.

original U is retained through the residual connection, and the corrected one-quarter-channel group U' is concatenated with the unchanged three-quarter-channel group V. The corrected group has size $U' \in \mathbb{R}^{B\times q\times H\times W}$, and channel concatenation produces a $B\times C\times H\times W$ output before the original C2f fusion. Zero-initializing γ makes the module an identity mapping at initialization.

C. Gated Interpolation Upsampling

Circular-knitting defects vary substantially in morphology and scale, from narrow horizontal and vertical patterns to small and large holes. Their feature representations therefore place different demands on spatial continuity, response concentration, and boundary extent. In the top-down feature pyramid, higher-level context must be reconstructed at a finer resolution before fusion with lateral features. Nearest-neighbor interpolation replicates responses within local regions, whereas bilinear interpolation combines neighboring responses to produce smoother spatial transitions. A fixed interpolation rule applies the same reconstruction behavior to all locations, limiting its adaptability to these diverse defect patterns. We develop GIU to adjust the balance between response replication and neighborhood smoothing according to the input features [Fig. 6(b)]. To achieve this adaptation with a small computational overhead, GIU combines two parameter-free interpolation paths using a single spatial weight map predicted at low resolution and shared across channels. It reconstructs the higher-level context before fusion with shallow lateral features, which retain local texture and localization cues.

1) Two Fixed Interpolation Paths: For $X \in \mathbb{R}^{B\times C\times H\times W}$, nearest-neighbor and bilinear upsampling produce

$$N = \mathrm{Nearest}_2(X) \tag{9}$$

$$L = \mathrm{Bilinear}_2(X) \tag{10}$$

where $N, L \in \mathbb{R}^{B\times C\times 2H\times 2W}$. Both operations use fixed interpolation weights and introduce no learnable parameters.

2) Shared Spatial Gate: A 3×3 convolution followed by Hardsigmoid predicts a low-resolution spatial weight map, which is then upsampled by nearest-neighbor interpolation:

$$g = \mathrm{HS}(\mathrm{Conv}_{3\times 3}(X)) \tag{11}$$

$$G = \mathrm{Nearest}_2(g) \tag{12}$$

where $\mathrm{Conv}_{3\times 3}$ denotes a biased convolution with C input channels, one output channel, and padding of 1, and $g \in [0,1]^{B\times 1\times H\times W}$ is the low-resolution weight map. The map $G \in [0,1]^{B\times 1\times 2H\times 2W}$ assigns a content-adaptive interpolation weight to each spatial location and is broadcast across feature channels. Predicting this map at $H\times W$ limits the spatial cost of the learned convolution, while channel sharing requires only one output weight per location.

3) Gated Reconstruction: The output $Y \in \mathbb{R}^{B\times C\times 2H\times 2W}$ is a convex combination:

$$Y = (1-G)\odot N + G\odot L \tag{13}$$

Small values of G favor response replication, while larger values favor neighborhood smoothing. This spatially varying mixture allows the reconstruction behavior to adapt to local

feature content associated with different defect morphologies and scales.

$$G_{\text{init}} = 0.05, \qquad Y_{\text{init}} = 0.95N + 0.05L \tag{14}$$

so the module starts from a predominantly nearest-neighbor reconstruction, consistent with the baseline top-down pathway.

IV. Experimental Setup

A. Dataset

The dataset was collected from a real circular-knitting production line and covers three defect categories: horizontal defects (*row*), vertical defects (*col*), and holes (*hole*). It comprises 403 images with 468 annotated defect instances, divided into training, validation, and test sets of 282, 40, and 81 images, respectively, as summarized in Table II.

TABLE II
IMAGE-LEVEL SPLIT AND DEFECT-INSTANCE COUNTS IN THE REAL PRODUCTION-LINE DATASET.

Split	Images	Instances	Row	Col	Hole
Train	282	329	174	75	80
Validation	40	48	24	13	11
Test	81	91	60	12	19

B. Implementation Details

TABLE III
IMPLEMENTATION PROTOCOL COVERING TRAINING (ULTRALYTICS) AND DEPLOYMENT (EDGE DEVICE).

Category	Parameter	Specification
Training setup	Hardware	Intel i7-13700KF; NVIDIA RTX 3090 (24 GB)
	Software	Ubuntu 24.04, Python 3.10.20, PyTorch 2.4.1+cu124, and Ultralytics 8.4.140
Deployment setup	Hardware	Rockchip RK3566
	System	Android 13
	Quantization	INT8
Data protocol	Input resolution	Fixed at 640×640
Training dynamics	Batch	16
	Maximum epochs	300
	Patience	80 epochs
	Optimizer	AdamW

Table III summarizes the training and deployment settings. MCGI-YOLO and its component-ablation variants share the same `yolov8n.pt` initialization as the YOLOv8n baseline, and all component comparisons use a common training protocol with matched random seeds. Other comparison models use available pretrained checkpoints from their respective implementations.

C. Evaluation Metrics

Intersection over union (IoU) measures the overlap between predicted and ground-truth bounding boxes as the ratio of their intersection area to their union area. Detection accuracy is evaluated using mAP50 and mAP50–95. Let $a_c(t)$ denote the average precision for class c at IoU threshold t. The metrics are computed as

$$\text{mAP}_{50} = \frac{1}{N_c} \sum_{c=1}^{N_c} a_c(0.50) \tag{15}$$

$$\text{mAP}_{50:95} = \frac{1}{10N_c} \sum_{c=1}^{N_c} \sum_{t \in T} a_c(t) \tag{16}$$

where $N_c = 3$ and $T = \{0.50, 0.55, \ldots, 0.95\}$. Both metrics are reported as percentages, with improvements expressed in percentage points.

Model parameters and FLOPs are measured at 640×640 input resolution using each model's respective framework. For FPS comparison, all models are evaluated on an RTX 3090 with FP32 precision, batch size 1, and 640×640 inputs. This measurement is separate from the RK3566 deployment result.

V. Results and Discussion

A. Comparison With Other Detectors

Table IV compares test-set accuracy, model complexity, and inference throughput. MCGI-YOLO achieves the highest mAP50 of 97.2% and the second-highest mAP50–95 of 65.5%, improving over the YOLOv8n [27] baseline by 6.7 and 7.1 percentage points, respectively. This gain comes with only a marginal increase in parameters (3.01 M $\rightarrow$ 3.02 M) and FLOPs (8.1 G $\rightarrow$ 8.2 G), while throughput on the RTX 3090 decreases from 554 to 486 FPS. Among lightweight variants, YOLO11n [27] attains the next-best accuracy (93.3% mAP50, 60.8% mAP50–95); MCGI-YOLO surpasses it by 3.9 and 4.7 points while running faster (486 vs. 463 FPS). YOLOv9t [31], YOLOv10n [32], YOLO12n [33], and YOLO26n [27] have fewer parameters or lower FLOPs but underperform on this test set, confirming that task-specific adaptation outperforms generic lightweight detectors. RT-DETR-R18 [34] leads in mAP50–95 (67.7%) but requires 20.09 M parameters and 60.7 G FLOPs at 146 FPS. MCGI-YOLO also outperforms DETR-R50 [35], Deformable DETR-R50 [36], and Faster R-CNN-R50 [37] on both mAP metrics with substantially lower cost, offering the best overall accuracy–efficiency tradeoff. Fig. 7 shows that MCGI-YOLO produces fewer fragmented boxes and tighter localization on elongated defects and holes, consistent with the aggregate results.

B. Component Ablation

Table V defines four configurations: the original YOLOv8n baseline, MCE alone, GIU alone, and their combination. The MCE-only model retains nearest-neighbor upsampling, while the GIU-only model retains the original backbone C2f blocks. On the test set, MCE alone achieves 93.9% mAP50 and 63.6% mAP50–95, improving over the baseline by 3.4 and 5.2 percentage points, respectively. GIU alone achieves 94.4% and 63.0%, corresponding to gains of 3.9 and 4.6 points. Combining the two modules yields 97.2% mAP50 and 65.5% mAP50–95, exceeding the best single-module result for each metric by 2.8 and 1.9 points. These results support the

TABLE IV
DETECTOR COMPARISON ON THE TEST SET. FPS DENOTES FP32 MODEL-FORWARD THROUGHPUT ON AN RTX 3090 WITH BATCH SIZE 1 AND A 640×640 INPUT. BOLD AND UNDERLINED VALUES INDICATE THE BEST AND SECOND-BEST RESULTS, RESPECTIVELY.

Model	Params (M)	FLOPs (G)	mAP50 (%)	mAP50–95 (%)	FPS
YOLOv9t [31]	**2.12**	8.5	90.1	53.3	291
YOLOv10n [32]	2.78	8.8	88.2	58.5	<u>524</u>
YOLO11n [27]	2.62	<u>6.7</u>	93.3	60.8	463
YOLO12n [33]	2.60	7.6	89.0	56.8	326
YOLO26n [27]	<u>2.57</u>	**6.2**	90.5	58.2	425
RT-DETR-R18 [34]	20.09	60.7	95.2	**67.7**	146
DETR-R50 [35]	41.28	77.3	96.4	53.6	101
Deformable DETR-R50 [36]	39.8	158.7	<u>96.8</u>	64.3	49
Faster R-CNN-R50 [37]	41.1	181.5	96.1	64.7	66
YOLOv8n (Baseline) [27]	3.01	8.1	90.5	58.4	**554**
MCGI-YOLO	3.02	8.2	**97.2**	<u>65.5</u>	486

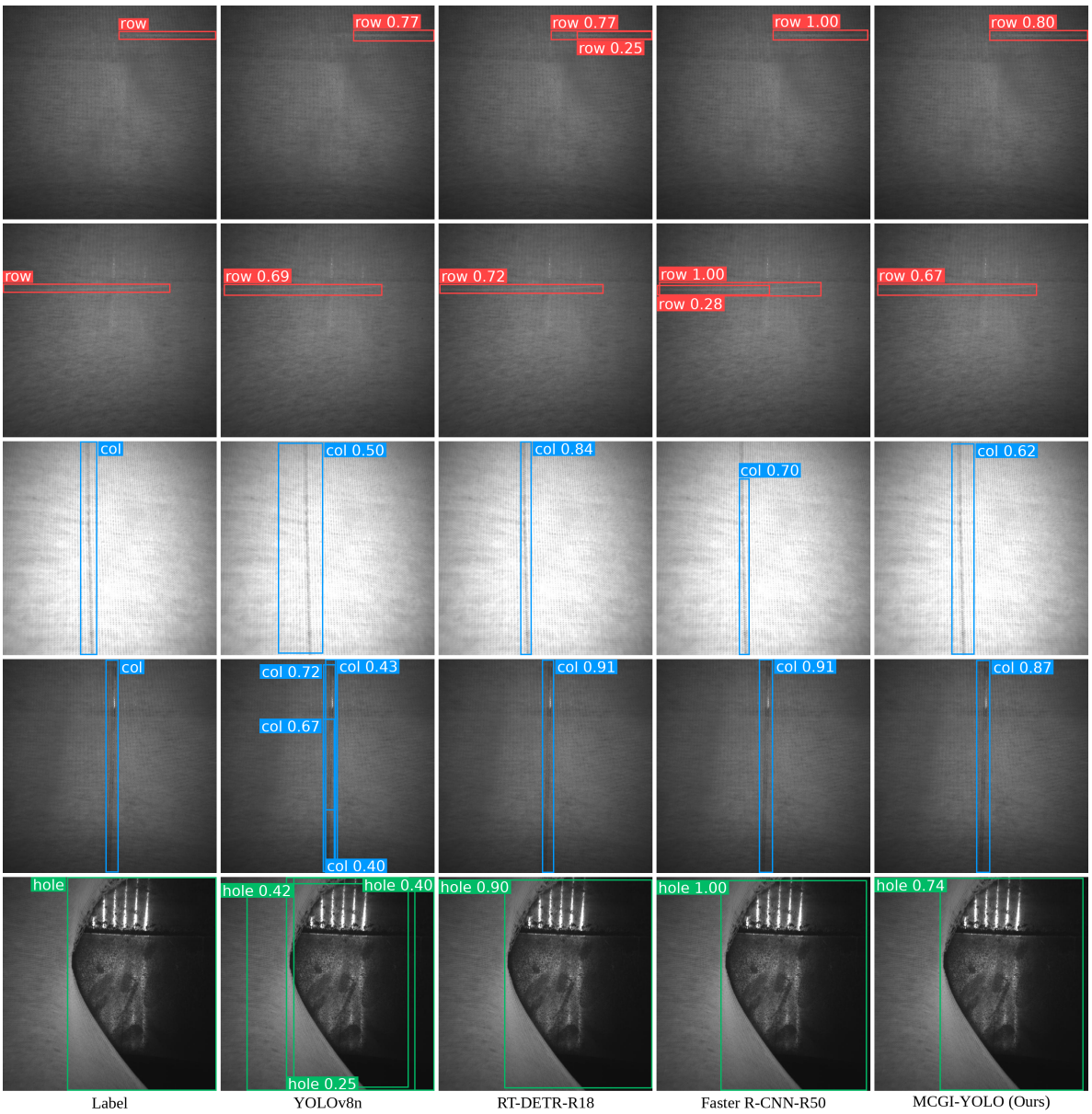

Fig. 7. Qualitative comparison of detection results across different methods. From left to right: ground-truth annotations (Label), YOLOv8n, RT-DETR-R18, Faster R-CNN-R50, and MCGI-YOLO (Ours).

TABLE V
COMPONENT ABLATION OF MCE AND GIU.

Variant	MCE	GIU	Params (M)	FLOPs (G)	mAP50	mAP50–95
YOLOv8n	–	–	3.01	8.10	90.5	58.4
+ MCE	✓	–	3.01	8.2	93.9	63.6
+ GIU	–	✓	3.01	8.2	94.4	63.0
MCGI-YOLO	✓	✓	3.02	8.2	97.2	65.5

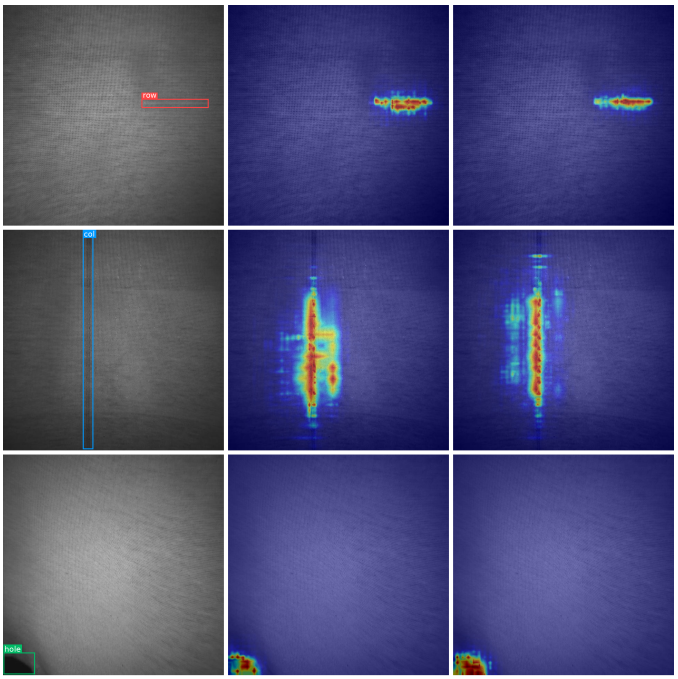

Fig. 8. Feature visualization of YOLOv8n and MCGI-YOLO. From left to right: ground-truth annotations (Label), YOLOv8n, and MCGI-YOLO.

complementary benefits of morphology-aware context encoding and adaptive feature reconstruction. Feature visualizations in Fig. 8 complement the numerical analysis by comparing responses at corresponding feature stages on the same images.

VI. SYSTEM DEPLOYMENT AND OPERATION

The trained detector is deployed on the dedicated edge device and integrated with the client interface, cloud platform, and configurable stopping rules. This section describes on-device inference performance and how detection outputs support local intervention and remote supervision.

A. Model Deployment and On-Device Performance

To deploy MCGI-YOLO on the RK3566 edge device, the trained model is exported to ONNX with a fixed input size of 640×640 and then converted into an INT8 RKNN model using RKNN-Toolkit2 [38]. For 640×640 inputs, the deployed INT8 model achieves a measured inference throughput of 22 FPS on the RK3566. This throughput is 2.2 times the 10 FPS design target derived in Section II-D, meeting the model-inference rate required for the specified machine speed and camera field of view.

B. Client Detection Interface

Fig. 9 shows the client interface during detection and after a defect is found. The home page displays the detection state, device information, and current sensitivity setting. Upon defect detection, the client automatically switches to the defect-detail page, displaying the captured image, the localized defect, and its category for operator review. This transition links the detection result to an accessible on-machine record for fault handling.

C. Cloud Platform and Digital Twin

The edge device reports defect events and runtime states to the cloud and receives remote configuration updates. Local detection and machine-stop commands are executed on the edge device, while the cloud provides centralized monitoring and management.

The IoT platform in Fig. 10 consolidates the uploaded information. Panel A) summarizes defect counts and defect events associated with machine stopping, while panel B) presents image-based defect records for individual devices. Panels C) and D) form the digital-twin interface for the edge client: the cloud displays the reported runtime state and provides controls that update the client's detection configuration. The state view includes the application version, camera status, CPU temperature, fabric type, and detection state. The control view allows remote adjustment of sensitivity and associated decision thresholds. Together, state synchronization and configuration feedback connect the physical client with its cloud-side representation, supporting remote supervision and control.

D. Configurable Machine-Stop Rules

Textile factories differ in their tolerance for fabric defects and the quality requirements of their products. The client therefore provides three preset sensitivity levels, low, medium, and high, with editable stopping thresholds for each defect category. Operators can select a preset and adjust its parameters to match the factory's acceptance criteria. The configuration can also be updated remotely through the interface in Fig. 10(D).

Each sensitivity preset combines a confidence threshold, three geometric thresholds, and a defect-count threshold per frame, as listed in Table VI. Predicted bounding boxes are mapped back to the original 1280×1024 camera image before their dimensions are evaluated. For sensitivity level s, let τ_s denote the confidence threshold, W_s and H_s the horizontal- and vertical-defect length thresholds in pixels, A_s the hole-area threshold in square pixels, and N_s the defect-count threshold. For detection i with class c_i, bounding-box width w_i, and height h_i, the geometric qualification indicator is

$$r_i^{(s)} = \begin{cases} \mathbb{I}(w_i > W_s), & c_i = \text{row} \\ \mathbb{I}(h_i > H_s), & c_i = \text{col} \\ \mathbb{I}(w_i h_i > A_s), & c_i = \text{hole} \end{cases} \tag{17}$$

where $\mathbb{I}$ equals 1 when the condition is satisfied and 0 otherwise. Width measures the horizontal extent of a row defect, height measures the vertical extent of a column defect, and bounding-box area measures the extent of a hole. With p_i denoting detection confidence, let $\mathcal{D}_s = \{i \mid p_i \geq \tau_s\}$ be the set

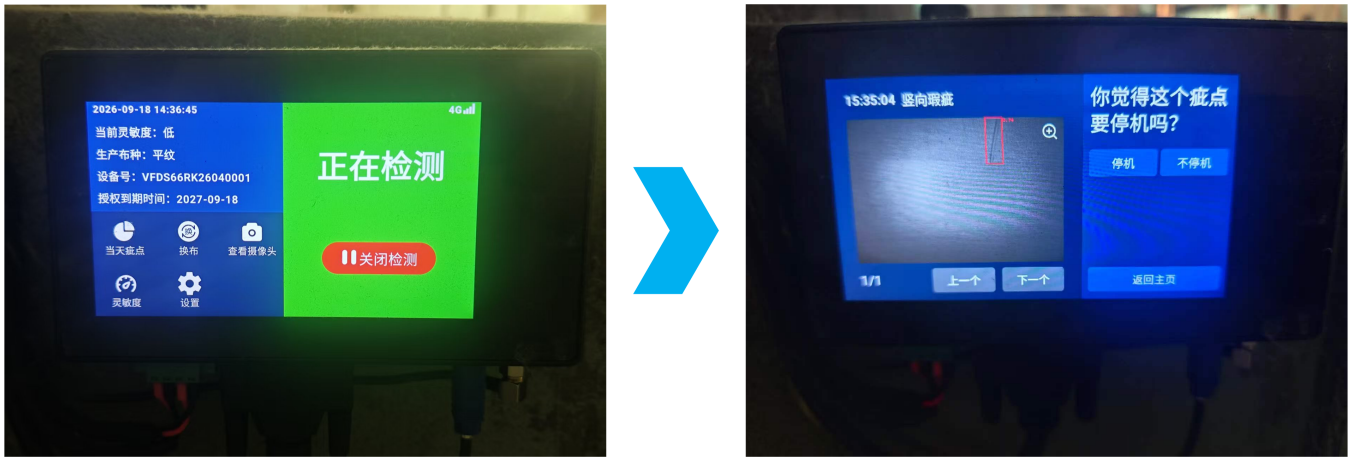

Fig. 9. Client interface transition upon defect detection: the detection home page (left) switches to the defect-detail page (right), showing the defect image, localization result, and category.

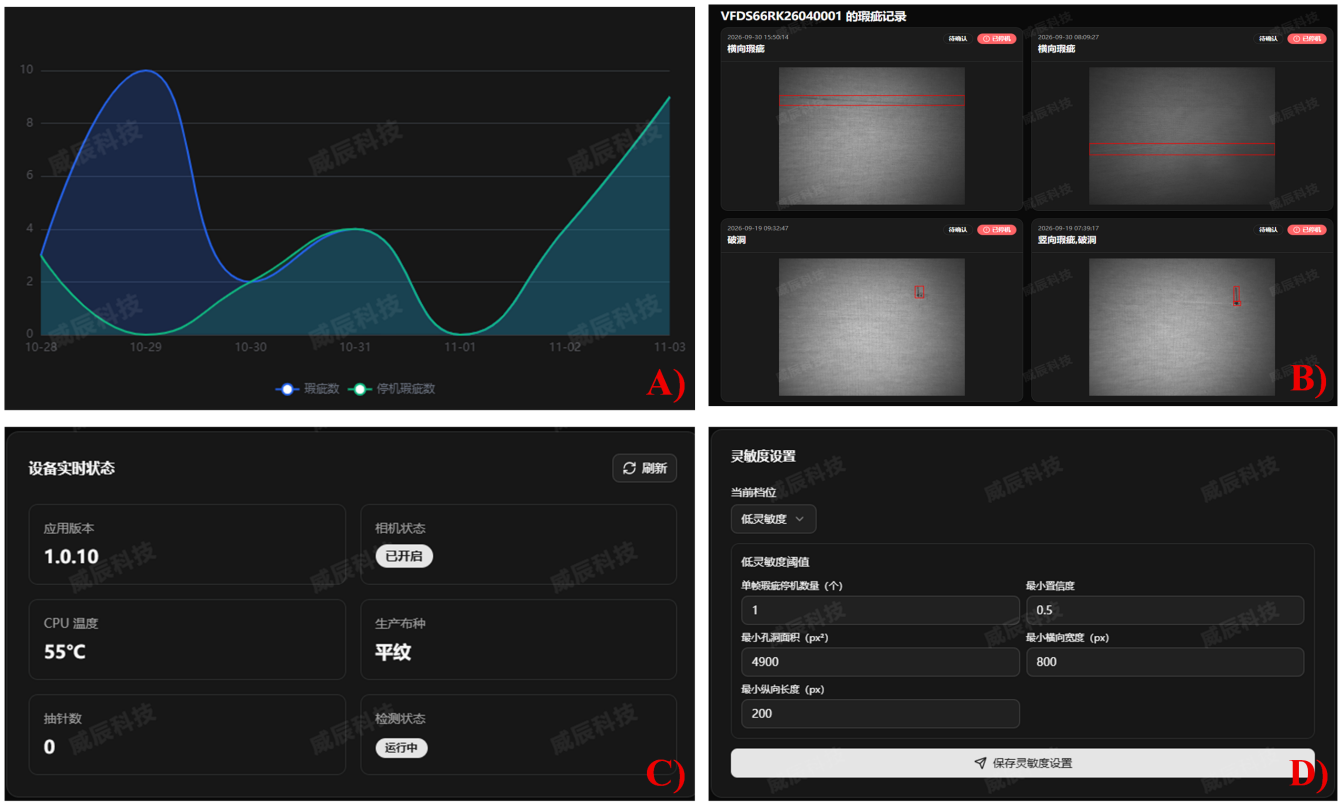

Fig. 10. IoT platform interfaces: A) defect-event statistics; B) device-specific defect records; C) digital-twin view of the client's runtime state; and D) digital-twin controls for remotely configuring client sensitivity and decision thresholds.

of confidence-qualified detections in the current frame. The stopping decision is

$$S_s = \mathbb{I}\left(\sum_{i\in\mathcal{D}_s} r_i^{(s)} \geq 1 \ \vee \ |\mathcal{D}_s| \geq N_s\right) \tag{18}$$

A stop command is issued when either a confidence-qualified detection exceeds its class-specific size threshold or the total number of confidence-qualified detections reaches N_s. The count criterion includes defects below the geometric thresholds, allowing multiple small defects to trigger stopping. Counts are evaluated independently for each frame.

TABLE VI
PRESET STOPPING PARAMETERS. GEOMETRIC THRESHOLDS USE THE ORIGINAL 1280×1024 IMAGE COORDINATES; N_s IS THE CONFIDENCE-QUALIFIED DEFECT-COUNT THRESHOLD PER FRAME.

Level	τ_s	A_s (pixels^2)	W_s (pixels)	H_s (pixels)	N_s
Low	0.50	6400	800	400	3
Medium	0.40	2500	400	200	2
High	0.30	1000	200	100	1

Higher sensitivity lowers the confidence, size, and count thresholds, allowing stopping for smaller defects or fewer detections within a frame. Lower sensitivity raises these thresholds to accommodate less stringent acceptance criteria. The editable presets convert detection outputs into factory-specific production decisions without retraining the detector.

E. Illustrative Analysis of Delayed Intervention

A persistent yarn abnormality at one active feeder can generate a new defective course on each machine revolution, following the course-formation mechanism of circular weft knitting [39]. Consider a hypothetical five-minute delay between defect onset and operator discovery. With the rotation period $T = 2$ s specified in Section II-D, the machine completes $N = 300/T = 150$ revolutions during this interval. If the abnormality persists throughout each revolution and each horizontal defect extends approximately around the fabric circumference, the diameter $D = 1.016$ m gives a cumulative transverse defect length of

$$L_{\text{sum}} = N\pi D = 150 \times \pi \times 1.016 \approx 478.8 \text{ m}. \tag{19}$$

Taking an approximate fabric take-down increment of $p = 0.03$ m per revolution, the affected fabric length in the production direction is

$$L_{\text{affected}} = Np = \frac{300}{2} \times 0.03 \approx 4.5 \text{ m}. \tag{20}$$

Under these operating assumptions, a five-minute discovery delay allows approximately 150 defective courses, with a cumulative transverse length of 479 m, to recur over a 4.5 m length of fabric in the production direction. These estimates connect intervention delay to the extent of affected production and illustrate the value of on-machine detection and timely stopping in limiting further defect propagation.

VII. Conclusion

This paper presents a dedicated defect detection and closed-loop control system for circular knitting machines, integrating image acquisition, SerDes transmission, edge inference, machine stopping, and cloud alerts. The lightweight MCGI-YOLO combines MCE for elongated-defect morphology encoding with GIU for adaptive feature reconstruction. On the self-collected test set, it achieves 97.2% mAP50 and 65.5% mAP50–95, improving over YOLOv8n by 6.7 and 7.1 percentage points. INT8 deployment on the RK3566 achieves 22 FPS, exceeding the 10 FPS design target. Unlike generic object detection pipelines, our work jointly tailors both the hardware architecture and the detection algorithm to the morphology of knitting defects and the constraints of edge deployment, enabling real-time closed-loop control directly on the machine. By delivering a fully integrated and deployable system rather than an isolated algorithm, this work addresses a critical bottleneck in textile manufacturing—namely, the difficulty of translating detection performance into actionable, on-machine quality control. It offers the industry a ready-to-deploy reference for intelligent detection of circular knitting machines and, more broadly, demonstrates a reproducible pathway for deploying lightweight vision models on resource-constrained edge devices in industrial settings.

References

[1] Cotton Incorporated, "Knit basics," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/knit-basics/. Accessed: Oct. 8, 2026.

[2] The Woolmark Company, "Circular knit technology," [Online]. Available: https://www.woolmark.com/industry/product-development/processing-innovations/circular-knitting-technology/. Accessed: Oct. 8, 2026.

[3] Cotton Incorporated, "Single and double knits," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/single-and-double-knits/. Accessed: Oct. 8, 2026.

[4] International Textile Manufacturers Federation, "Global textile machinery shipments shrunk in 2025, except for spinning," Jul. 2026, [Online]. Available: https://www.itmf.org/images/dl/press-releases/2026/2026-07-24-ITMSS-48-2025.pdf. Accessed: Oct. 10, 2026.

[5] United Nations Environment Programme, "Transforming the textile industry: Learning to manage waste, better," United Nations Environment Programme, Tech. Rep., Oct. 2025, [Online]. Available: https://wedocs.unep.org/handle/20.500.11822/48707. Accessed: Oct. 14, 2026.

[6] S. Takeuchi, K. Nishioka, H. Uematsu, and S. Tanoue, "Research into development of the defect detection system for knitted fabric produced by the circular knitting machines by image analysis," *Journal of Textile Engineering*, vol. 64, DOI 10.4188/jte.64.45, no. 2, pp. 45–49, 2018.

[7] M. M. K. Akter, U. N. Haq, M. M. Islam, and M. A. Uddin, "Textile-apparel manufacturing and material waste management in the circular economy: A conceptual model to achieve sustainable development goal (SDG) 12 for bangladesh," *Cleaner Environmental Systems*, vol. 4, DOI 10.1016/j.cesys.2022.100070, p. 100070, 2022.

[8] Z. Xiang, Y. Shen, M. Ma, and M. Qian, "HookNet: Efficient multiscale context aggregation for high-accuracy detection of fabric defects," *IEEE Trans. Instrum. Meas.*, vol. 72, DOI 10.1109/TIM.2023.3280519, pp. 1–11, 2023, art. no. 5016311.

[9] R. Machado, L. A. M. Barros, V. Vieira, F. D. da Silva, H. Costa, and V. Carvalho, "Textile defect detection using artificial intelligence and computer vision—a preliminary deep learning approach," *Electronics*, vol. 14, DOI 10.3390/electronics14183692, no. 18, p. 3692, 2025.

[10] T. Yan, Y. Wu, Y. Shi, C. Lin, and L. Ji, "Development and application of an AI visual defect detection system for warp-knitted lace based on 5G+ technology," *Information*, vol. 17, DOI 10.3390/info17070623, no. 7, p. 623, 2026.

[11] Y. Wang, H. Yu, B. Guo, H. Shi, and Z. Yu, "Research on real-time detection system of rail surface defects based on deep learning," *IEEE Sensors J.*, vol. 24, DOI 10.1109/JSEN.2024.3402730, no. 13, pp. 21 157–21 167, 2024.

[12] Y. Zhang, X. Xue, L. Mu, J. Xin, J. Huang, and Y. Zhang, "UAV-based edge inference for insulator defect detection on transmission lines via convolutional feature reuse," *IEEE Internet Things J.*, vol. 13, DOI 10.1109/JIOT.2026.3681637, no. 12, pp. 27 790–27 802, 2026.

[13] D. Li, Y. Lu, Q. Gao, X. Li, X. Yu, and Y. Song, "LiteYOLO-ID: A lightweight object detection network for insulator defect detection," *IEEE Trans. Instrum. Meas.*, vol. 73, DOI 10.1109/TIM.2024.3418082, pp. 1–12, 2024, art. no. 5023812.

[14] B. Zhao, Y. Wang, H. Zhang, J. Liu, and J. Zhang, "KAM-Net: Kilobyte-scale ultralightweight attention-based network for glass defect detection with algorithm/hardware co-design," *IEEE Trans. Instrum. Meas.*, vol. 74, DOI 10.1109/TIM.2025.3593602, pp. 1–10, 2025, art. no. 5041310.

[15] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 779–788, 2016.

[16] K. Yu, W. Lyu, X. Yu, Q. Guo, W. Xu, and L. Zhang, "FA-YOLO: A high-precision and efficient method for fabric defect detection in textile industry," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E107.A, DOI 10.1587/transfun.2023EAP1030, no. 6, pp. 890–898, 2024.

[17] Z. Zhang, T. Huang, W. Zhang, Y. Yang, and Q. Yu, "DGHR-YOLO: Fabric defect detection based on high-level screening-feature pyramid networks and deformable convolution," *Nondestructive Testing and Evaluation*, vol. 40, DOI 10.1080/10589759.2025.2451768, no. 12, pp. 5919–5944, 2025.

[18] Y. Zhao, Q. Liu, H. Su, J. Zhang, H. Ma, W. Zou, and S. Liu, "Attention-based multiscale feature fusion for efficient surface defect detection," *IEEE Trans. Instrum. Meas.*, vol. 73, DOI 10.1109/TIM.2024.3372229, pp. 1–10, 2024, art. no. 5013310.

[19] N. An and L. Wan, "JDCBL: A joint dynamic contexts and balanced learning framework for real-world fabric defect detection," *IEEE Trans. Instrum. Meas.*, vol. 74, DOI 10.1109/TIM.2025.3558828, pp. 1–12, 2025, art. no. 5026412.

[20] H. Wang, B. Chen, C. Xie, and W. Ye, "Real-time oriented fabric defect detection via position-guided fusion and multiscale rotation-guided strip coupling," *IEEE Trans. Ind. Informat.*, DOI 10.1109/TII.2026.3717200, pp. 1–12, 2026, early Access.

[21] B. Su, H. Chen, and Z. Zhou, "BAF-Detector: An efficient CNN-based detector for photovoltaic cell defect detection," *IEEE Trans. Ind. Electron.*, vol. 69, DOI 10.1109/TIE.2021.3070507, no. 3, pp. 3161–3171, 2022.

[22] X. Shen, J. Liu, L. Jiang, X. Liu, and H. Zhang, "A novel weld defect detection method for intelligent magnetic flux leakage detection system via contextual relation network," *IEEE Trans. Ind. Electron.*, vol. 71, DOI 10.1109/TIE.2023.3294578, no. 6, pp. 6304–6314, 2024.

[23] J. Liu, Z. Wen, X. Shen, F. Zuo, L. Jiang, and H. Zhang, "Online pipeline weld defect detection for magnetic flux leakage inspection system via lightweight rotated network," *IEEE Trans. Ind. Electron.*, vol. 72, DOI 10.1109/TIE.2024.3503635, no. 7, pp. 7573–7584, 2025.

[24] THine Electronics, Inc., "THCV241A: MIPI CSI-2 to V-by-One HS serializer," [Online]. Available: https://www.thine.co.jp/en/products/detail/THCV241A.html. Accessed: Oct. 16, 2026.

[25] THine Electronics, Inc., "THCV242A: V-by-One HS to MIPI CSI-2 deserializer," [Online]. Available: https://www.thine.co.jp/en/products/detail/THCV242A.html. Accessed: Oct. 16, 2026.

[26] Rockchip Electronics, "RK3566 brief datasheet," [Online]. Available: https://www.rock-chips.com/uploads/pdf/2022.8.26/191/RK3566%20Brief%20Datasheet.pdf. Accessed: Oct. 15, 2026.

[27] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.

[28] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, DOI 10.1109/CVPR.2017.106, pp. 2117–2125, 2017.

[29] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, DOI 10.1109/CVPR.2018.00913, pp. 8759–8768, 2018.

[30] Q. Hou, D. Zhou, and J. Feng, "Coordinate attention for efficient mobile network design," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 13 713–13 722, 2021. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2021/html/Hou_Coordinate_Attention_for_Efficient_Mobile_Network_Design_CVPR_2021_paper.html

[31] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. Eur. Conf. Comput. Vis.*, DOI 10.1007/978-3-031-72751-1_1, pp. 1–21, 2025.

[32] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," in *Adv. Neural Inf. Process. Syst.*, vol. 37, DOI 10.52202/079017-3429, pp. 107 984–108 011, 2024.

[33] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," in *Adv. Neural Inf. Process. Syst.*, vol. 38, DOI 10.52202/085713-2627, 2025.

[34] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, DOI 10.1109/CVPR52733.2024.01605, pp. 16 965–16 974, 2024.

[35] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, pp. 213–229, 2020.

[36] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.

[37] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.

[38] Rockchip, "Rknn toolkit2," [Online]. Available: https://github.com/airockchip/rknn-toolkit2, 2026, accessed: 2026-09-27.

[39] Cotton Incorporated, "Knit machinery: Flat, circular, and seamless," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/knit-machinery/. Accessed: Nov. 3, 2026.

Kangzhao Tan received the B.S. degree in computer science and technology from Guangzhou University, Guangzhou, China, in 2022. From 2022 to 2024, he worked as a cloud computing R&D engineer with Sangfor Technologies Inc., Shenzhen, China. He is currently pursuing the M.S. degree in computer technology with the School of Computer Science, Guangdong University of Technology, Guangzhou, China.

His research interests include deep learning, computer vision, and industrial defect detection.

Jiarui Zhang received the B.S. degree in electrical engineering and automation from Guangdong Polytechnic Normal University, Guangzhou, China, in 2021. He is currently pursuing the M.S. degree in software engineering with the School of Computer Science, Guangdong University of Technology, Guangzhou, China.

His research interests include electronic circuit design and intelligent Internet of Things.

Junjie Yu received the B.S. degree in computer science and technology from Hunan University of Finance and Economics, Changsha, China, in 2025. He is currently pursuing the M.S. degree in computer technology at Guangdong University of Technology, Guangzhou, China.

His research interests include image processing and edge computing.

Heng Wu received the B.S. degree in electronic and information engineering from Wuhan University of Science and Technology, Wuhan, China, in 2009, and the M.S. degree in optics and the Ph.D. degree in mechanical manufacturing and automation from South China University of Technology, Guangzhou, China, in 2012 and 2017, respectively.

He is currently an associate professor with the School of Automation, Guangdong University of Technology. His research interests include optical imaging, optical measurement, machine vision, and image processing.

Jing Lin received the B.S. degree in computer science and technology and the M.S. degree in software engineering from Guangdong University of Technology, Guangzhou, China, in 2003 and 2006, respectively.

Since graduation, he has served as an assistant engineer, engineer, and senior engineer. He is currently a senior engineer with the Classroom Management Center, Guangdong University of Technology. His research interests include information system project management, system informatization, and the intelligent Internet of Things.

Diyu Li received the B.S. degree from South China Agricultural University, Guangzhou, China, in 1996, and the M.S. degree in business administration from Lanzhou University, Lanzhou, China, in 2009.

In 2017, he founded Guangzhou Vinsome Automation Technology Co., Ltd., Guangzhou, China. His research interests include artificial intelligence and industrial automation.

Chunhua He received the B.S. degree in microelectronics from Sun Yat-sen University, Guangzhou, China, in 2010, and the M.S. and Ph.D. degrees in microelectronics from Peking University, Beijing, China, in 2013 and 2018, respectively.

From 2013 to 2017, he was an engineer with the No. 5 Electronics Research Institute, Ministry of Industry and Information Technology, Guangzhou, China. From 2017 to 2019, he was an engineer with Midea Group, Foshan, China. From 2019 to 2021, he was a senior engineer with Guangzhou 37 Degree Smart Home Co., Ltd., Guangzhou, China. He joined the School of Computer Science, Guangdong University of Technology, Guangzhou, China, in 2021, where he is currently a professor. His research interests include the design and application of MEMS sensors and artificial intelligence algorithms.